

Research paper

Enhancing Autonomous Vehicle control in complex scenarios with Deep Transformer Reinforcement Learning

Badr Ben ELallid ^{a, b, *}, Nabil Benamar ^{a, b}, Miloud Bagaa ^c, Nabil Mrani ^a

^a Moulay Ismail University, Meknes, Morocco

^b Al Akhawayn University, Ifrane, Morocco

^c Department of Electrical and Computer Engineering, University of Québec at Trois-Rivières, Trois-Rivières, QC, Canada

ARTICLE INFO

Keywords:

Autonomous vehicles
Reinforcement learning
Vehicle control
Transformer

ABSTRACT

Autonomous driving (AD) aims to significantly enhance road safety and driving comfort, contributing to a substantial reduction in traffic-related fatalities. With the rapid advancement of Artificial Intelligence (AI), particularly Deep Reinforcement Learning (DRL), the control of Autonomous Vehicles (AVs) has seen notable progress. While traditional approaches often rely on Convolutional Neural Networks (CNNs) within Deep Q Networks (DQNs) to process AV state information, they typically struggle with capturing long-range dependencies and often overlook the dynamic presence of diverse road users, such as pedestrians, cyclists, and motorcyclists. In this study, we propose a novel DRL model based on Deep Transformer Q Networks (DTQNs) to address these limitations. Our model integrates Transformer-based architectures to better model temporal and spatial dependencies in complex driving environments. Moreover, it explicitly accounts for the behavior of surrounding road participants to ensure safer and more efficient navigation. We evaluate our approach using the CAR Learning to Act (CARLA) simulator in scenarios involving heavy traffic and mixed road users. Experimental results demonstrate that our DTQN model significantly outperforms traditional DQN methods by reducing both travel delays and collision rates, thereby highlighting its potential for real-world autonomous driving applications.

1. Introduction

Intelligent Transport Systems (ITS) represent a cutting-edge frontier in road safety and convenience. They empower drivers with informed choices, aiming to enhance road safety, reduce traffic accidents, and improve the overall driving experience. At its core, ITS is committed to making our roadways safer and more efficient.

The World Health Organization (WHO) reports a shocking 1.3 million annual traffic-related fatalities, with a significant portion being pedestrians, cyclists, and motorcyclists (Be, 2021). AVs companies are on a mission to reduce these numbers in the future. AVs hold the promise of significantly reducing accidents, particularly those caused by human errors. They combine the skills of experienced human drivers with a primary focus on safety, benefiting all road users, including Vulnerable Road Users (VRUs) (El Hamdani et al., 2020b,a; Elallid et al., 2022c).

In the current mobility revolution era, AVs collaborate by sharing critical information with each other, road infrastructure, and with VRUs (Lamssaggad et al., 2021). However, navigating complex scenarios remains challenging, especially in dense traffic. Efficient vehicle

control systems capable of assessing rapidly changing road conditions are essential. In this context, AI techniques, such as Deep Learning (DL) and DRL, play a pivotal role in addressing these challenges (Elallid et al., 2023; Bao et al., 2023).

DL excels at understanding the environment and identifying objects on the road, enhancing safety (Hu et al., 2022). DRL methods excel in tasks like motion planning, decision-making, and vehicle control. They enable agents to learn from their surroundings through exploration and exploitation by optimizing various actions. Despite the significant progress made in AVs, there is a gap in research concerning the control of AVs in extremely complex scenarios with dense traffic. The effectiveness of AI algorithms in these challenging situations needs further investigation. As urban areas become more congested, traditional traffic management systems struggle to cope with complexity and unpredictability. ML, particularly DRL, offers the potential for intelligent vehicle control systems capable of adapting to dynamic environments in real time.

DRL combines deep neural networks with Reinforcement Learning (RL) allows autonomous agents to develop optimal strategies through

* Correspondence to: Computer Sciences at the School of Technology, Moulay Ismail University of Meknes, Meknes 50050, Morocco.

E-mail addresses: badr.benelallid@edu.umi.ac.ma (B.B. ELallid), n.benamar@umi.ac.ma (N. Benamar), miloud.bagaa@uqtr.ca (M. Bagaa), n.mrani@umi.ac.ma (N. Mrani).

<https://doi.org/10.1016/j.engappai.2025.111483>

Received 12 December 2023; Received in revised form 24 May 2025; Accepted 9 June 2025

Available online 24 June 2025

0952-1976/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

trial and error. In the realm of vehicle control, DRL has the potential to revolutionize how vehicles navigate traffic, make crucial decisions, and handle unforeseen challenges. By harnessing DRL's power, we can create advanced control systems that adapt autonomously to changing traffic conditions, ushering in an era of improved traffic efficiency and safety.

Self-attention-based models, particularly Transformers (Vaswani et al., 2017), have emerged as the dominant approach in natural language processing. To leverage the strengths of Transformers, some studies (Stigall, 2024; Esslinger et al., 2022) have integrated RL with Transformer architectures, enhancing the DQN algorithm by replacing the traditional CNN with a Transformer in RL-based game environments. However, no existing work has explored the combination of CNN and Transformer to address intersection challenges in AVs.

One notable limitation of existing studies is the lack of dedicated research focused on enhancing the perception capabilities of AVs, which are critical for understanding their surroundings and making informed decisions. Moreover, incorporating temporal information into the training of RL agents is essential, as it allows them to utilize past states to make better decisions. Additionally, many previous works address simplified scenarios, often overlooking the presence of other road users, such as pedestrians, vehicles, and cyclists. To overcome these challenges, we propose a novel approach aimed at enabling effective AV control in complex scenarios, particularly those involving dense traffic.

The motivation behind combining CNNs, DQN, and transformers in this work lies in their complementary strengths, which address key challenges in AV perception and decision-making. CNNs excel at extracting detailed local features from visual data such as edges and textures making them ideal for detecting nearby objects and obstacles. However, CNNs alone struggle to capture long-range dependencies and global context. Transformers, by contrast, are well suited for modeling global relationships, enabling a deeper understanding of the driving environment, such as identifying distant objects or interpreting complex traffic patterns. By integrating these architectures within a RL framework like DQN, the system can combine precise local feature extraction with a comprehensive global perspective to make more informed decisions.

In summary, this paper presents three innovative contributions that are highly convenient:

1. We introduce a novel approach, DTQN, to enhance AV control, enabling the vehicle to navigate intersections with dense traffic and other road users effectively.
2. To address long-term temporal dependencies, we integrate a Transformer into the DQN algorithm. This enhancement helps identify and prioritize critical regions within images, thereby improving decision-making capabilities.
3. Our proposed model outperforms traditional DQN by reducing travel delay and the number of collisions.

The remainder of the paper is organized as follows: In Section 2, we review prior research, while Section 3 introduces the key concepts of DRL and the Transformer model. Section 4 delves into our novel DTQN approach for vehicle control. Section 5 outlines our simulation setup, and Section 6 presents experimental results. Finally, in Section 7, we summarize our findings and conclusions.

2. Related work

Vehicle control plays a crucial role in rectifying possible errors arising from motion planning and decision-making. In AVs, the controller's primary function is to ensure that the vehicle remains on course and stable. This is achieved by leveraging vehicle state information provided by perception and localization modules (Elallid et al., 2022a). Leading companies such as Waymo and Tesla rely on these methods

due to their reliability (Constantinides et al., 2024). Before control methods can be deployed, the perception system must handle fundamental tasks such as detecting relevant objects, including vehicles, pedestrians, and road environments (Pravallika et al., 2024). However, these pre-planning stages can be time-intensive, leading to delayed reaction times in critical scenarios. Moreover, conventional methods struggle in environments that deviate from predefined mathematical models and regulatory constraints.

Recent advancements in ML have sparked increasing interest in end-to-end learning approaches (Chen et al., 2024). These methods leverage sensor data, such as visual images from cameras, which are directly fed into a vehicle control model based on neural networks. Imitation learning (IL) has been widely employed (Guo et al., 2023), demonstrating strong performance in familiar scenarios but struggling with generalization due to dataset limitations.

DRL techniques, particularly DQN, have shown promising results in AD research. Elallid et al. (2023b) proposed a DQN-based approach to train AVs in complex traffic environments. Their method demonstrated improvements in travel efficiency and collision avoidance. However, their work relied on flattened image representations rather than feature extraction through convolutional networks.

Similarly, Elallid et al. (2022b) utilized DQN to enhance AV control in pedestrian-interactive scenarios. Their method significantly improved AV performance but suffered from suboptimal feature extraction. Pérez-Gil et al. (2021) employed CNNs in CARLA for vehicle control but ignored temporal dependencies and considered only static obstacles.

Nugroho et al. (2024) applied DQN for highway navigation in high-traffic conditions. While effective, their approach exhibited slow convergence and lacked generalization to unseen traffic patterns. Wang et al. (2024) used DQN for vehicle merging in connected autonomous vehicle (CAV) scenarios, though their method remained constrained by predefined control rules. Sivayazi and Mannayee (2024) introduced a DDQN-based urban navigation framework, but it accounted only for vehicles, omitting interactions with pedestrians and cyclists.

To address the limitations of conventional DQN models, researchers have explored hybrid architectures incorporating CNNs and Long Short-Term Memory (LSTM) networks. Ahmed et al. (2021) proposed a DQN-LSTM system for lane-keeping and safe distance maintenance but required further validation in dense traffic scenarios. Chen et al. (2020) introduced a Conditional DQN (CDQN) that fused CNNs with LSTMs and fuzzy logic, improving path stability. Li et al. (2022) developed a deep DQN model integrating CNN-LSTM for intersection navigation, enhancing decision-making accuracy.

Wu et al. (2023a) introduced a human-guidance-based DRL system that enabled real-time intervention during training, significantly improving learning efficiency. Selvaraj et al. (2023) proposed a DRL-based Adaptive Cruise Control (ACC) system, achieving notable improvements in headway performance and traffic flow management. Huang et al. (2023) designed a predictive behavior planning framework incorporating inverse reinforcement learning (IRL), achieving state-of-the-art performance in human-like decision-making.

Li et al. (2023) present a framework for safe motion planning in AVs by quantifying uncertainties in deep learning-based perception. Their approach introduces a probabilistic 3D object detector and a motion planner, PU-RRT, which together generate safe trajectories. Experimental results show a significant improvement in safety compared to existing methods.

Similarly, Chen et al. (2022) address the safety risks associated with learning-enabled control (LEC) in AVs. They propose a Simplex-based runtime assurance framework that switches to a verified safe controller using reachability analysis when the LEC is deemed unsafe. Although this method improves safety in simulations, it relies heavily on accurate system modeling, and the fallback controller may be overly conservative.

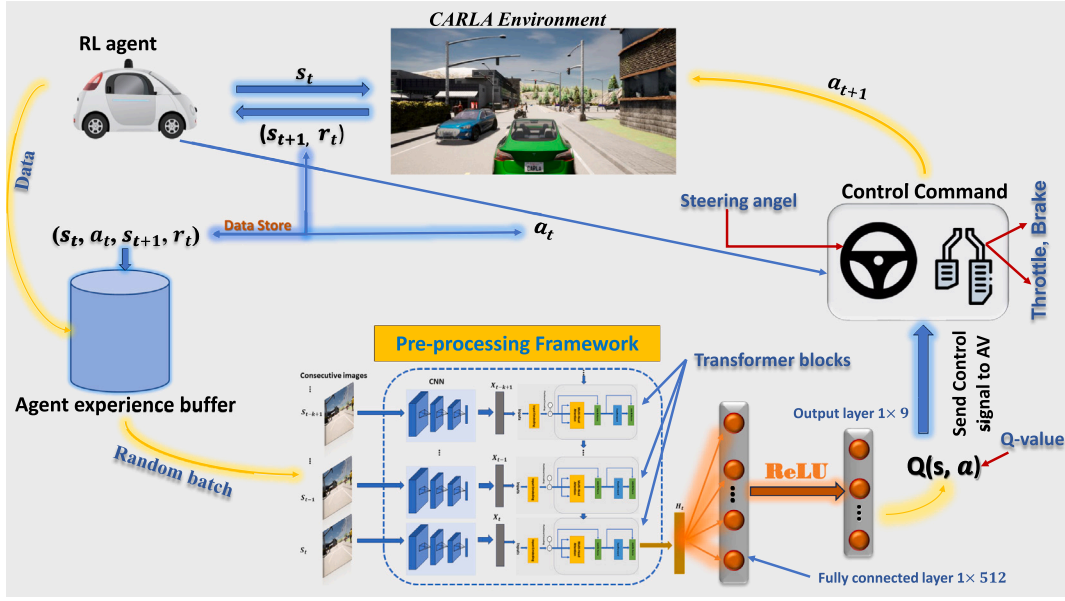


Fig. 1. Our pipeline begins with a RL agent interacting with the CARLA environment. The agent receives a state s_t and takes an action a_t to move to the next state s_{t+1} . The state consists of sequential images processed through a Pre-processing framework and a Transformer block. The final step involves FC layer to predict the Q-value of the action, enabling control of the AV within the environment.

In addition, Wang and Althoff (2023) proposes a safety shield based on online reachability analysis to prevent unsafe RL actions in AVs, thereby enhancing safety during the learning process.

Several studies have addressed the challenges of improving both ride comfort and energy efficiency in AVs and broader transportation systems. For instance, Du et al. (2022) propose a speed control framework for AVs utilizing DRL to enhance ride comfort and energy efficiency. By mitigating the effects of uneven road surfaces, the authors introduce the concept of a “maximum comfortable speed”. Experimental results demonstrate improvements of 8.22% in vertical ride comfort, 24.37% in energy efficiency, and 94.38% in computational efficiency compared to traditional optimization-based models.

Zhang et al. (2024) present a hierarchical Multi-Agent DRL approach based on Expert-MATD3 to optimize AV control strategies in urban environments. Their method simultaneously addresses energy efficiency, traffic flow, and safety. Experimental findings show a 10% reduction in energy consumption and a 20.37% decrease in travel time compared to baseline methods.

Wu et al. (2023b) focus on energy-efficient driving for autonomous electric vehicles operating in dynamic traffic conditions. They propose a DRL framework based on Double Dueling Deep Q-Networks, integrating both visual and physical inputs along with a safety-checking mechanism. The approach achieves a 23.3% improvement in energy efficiency while maintaining safety and travel speed.

Prior research in DRL-based AV control predominantly focuses on CNNs for feature extraction, with some models incorporating LSTMs for temporal awareness. However, LSTMs suffer from vanishing gradient issues, limiting their ability to capture long-term dependencies. In contrast, transformers utilize self-attention mechanisms, enabling the model to consider any position in the input sequence when making predictions. Given their success in computer vision and time-series data analysis, we integrate transformers into the DQN framework to enhance AV decision-making capabilities. It is important to note that this work does not consider ride comfort or energy efficiency. Our work advances DRL-based AV control by integrating transformers into the DQN framework. This provides enhanced generalization and adaptability, making AVs more capable of handling complex, dynamic environments beyond predefined scenarios.

3. Overview of deep reinforcement learning and transformer

To enhance the performance of DRL pipelines, especially in the context of AVs, it is crucial to merge the strengths of CNNs and Transformers. CNNs excel at capturing local and spatial features in images, while Transformers are proficient in modeling global context and long-range dependencies. By combining these two techniques, we can adopt a hierarchical approach that leverages the respective strengths of CNNs and Transformers.

3.1. Deep reinforcement learning

In recent years, there has been notable advancement in DRL, leading to its increased utilization across various domains, particularly in robotics (Shin et al., 2019). Previous studies have demonstrated the potential of the model-free DRL approach in enabling autonomous learning of motion control strategies, which has prompted significant dedication to the development of DRL-based methods. A RL algorithm is primarily based on the concept of a Markov decision process (MDP), which assumes that the current state of the system only depends on the previous state and not on the specific trajectory leading to it. MDPs are typically represented as a tuple (S, A, R, P) , where S represents the environmental state space in which the controller needs to make decisions, A is the set of available actions, R denotes the rewards or feedback obtained from taking actions at each time step, and P represents the probability distribution of state transitions.

In a typical MDP control task, the agent observes a state $s_t \in S$ at time t , selects an action $a_t \in A$, and the state transition probability P determines the next observed state s_{t+1} , as well as the rewards r_{t+1} obtained as a consequence of the action a_t taken. The agent’s objective is to maximize the expected return, $\mathbb{E}[G_t]$, where $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^T r_{t+T} = \sum_{k=0}^T \gamma^k r_{t+k}$ is the sum of discounted future rewards. Here, γ is a discount rate parameter ($0 < \gamma < 1$) that determines the relative importance assigned to immediate rewards compared to future rewards. A smaller γ value indicates a higher focus on immediate rewards rather than long-term ones. T represents the terminal state, but for continuous tasks, $T = \infty$, and the sum converges to a finite value due to the constraint $\gamma < 1$.

RL algorithms can be classified into three main types: value-based, policy-based, and hybrid actor-critic. Value-based algorithms focus on

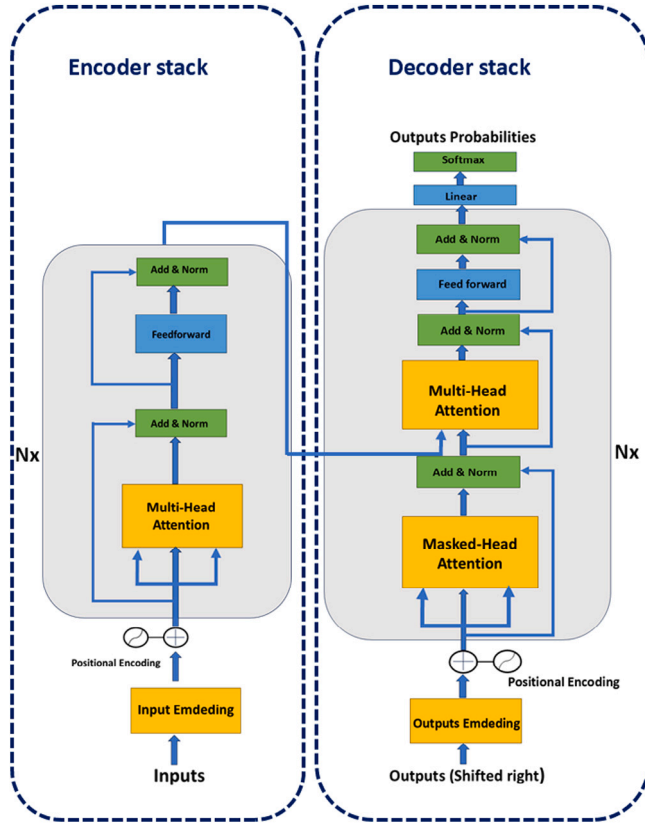


Fig. 2. The architecture of Transformer consists of an encoder and a decoder stack.

developing a value function, which indirectly determines the policy based on the learned value-function. The value-function represents the expected return or $\mathbb{E}[G_t]$. On the other hand, policy-based algorithms directly map states to actions based on feedback from the environment. Hybrid actor-critic methods combine both value-based and policy-based approaches. In this case, the policy is known as the actor, responsible for selecting actions, while the critic estimates the value-function and evaluates the actions taken by the actor.

3.2. Transformer

The Transformer, as introduced by Vaswani et al. in their seminal work (Vaswani et al., 2017), represents a groundbreaking advancement in neural network architecture. It was designed to address the limitations of traditional Recurrent Neural Networks (RNNs) and encoder-decoder frameworks. The Transformer achieves this by replacing RNNs with attention mechanisms, creating a sequence-to-sequence model with remarkable long-term memory capabilities.

The original Transformer model consists of a total of 12 layers arranged in two stacks: a 6-layer encoder stack on the left and a 6-layer decoder stack on the right as represented in Fig. 2. The output from one layer serves as the input for the next layer, ultimately leading to the final prediction.

This novel architecture operates by attentively considering every token in its history, granting it unparalleled contextual understanding. At its core, the Transformer consists of stacked components, including position-wise feed-forward layers, residual connections, and normalization layers, all built upon multiple multi-head attention layers.

To delve deeper into its functioning, we first need to grasp the fundamental concept of attention mechanisms within neural networks, which plays a pivotal role in the Transformer's operation.

Attention Mechanism: The Transformer leverages an attention mechanism to selectively focus on relevant information during processing. It employs the decoder state (referred to as Q : query) and the encoder hidden states (K : keys) to calculate attention weights. These weights assess the historical significance of encoder hidden states (V : values) concerning the decoder state. This computation is encapsulated within a generalized attention model, denoted as $Attention(Q, K, V)$, and expressed through Eq. (1):

$$Attention(Q, K, V) = F_{distribution}(F_{alignment}(K_i, Q)) \cdot V_i, \quad (1)$$

whereby:

- **Alignment score ($F_{alignment}$):** This function determines the similarity between the query and each key, quantifying the relevance of each key to the current query.
- **Distribution Function ($F_{distribution}$):** The alignment scores undergo transformation via a distribution function, often a softmax operation, to yield a probability distribution. This distribution indicates the extent of attention each element in the input sequence should receive.

Ultimately, the attention mechanism produces a weighted sum of the values, with each value scaled by its corresponding attention weight. This weighted sum serves as the output of the attention mechanism, which then feeds into subsequent layers within the Transformer.

Crucially, the Transformer does not employ a single attention mechanism but instead incorporates multiple parallel attention mechanisms, termed “multi-head attention”. Each head specializes in learning distinct representations of relationships between queries, keys, and values. This diversification enables the model to capture various types of dependencies in the data. These multiple heads are concatenated and linearly transformed to yield the final output.

Complementing multi-head attention, Transformer layers also encompass feed-forward layers, residual connections, and normalization layers. These components collaboratively empower the model to effectively process sequential data, preserving its capacity to capture both short-range and long-range dependencies. It is this unique combination of features that has rendered the Transformer architecture highly successful in a multitude of Natural Language Processing (NLP) tasks.

From other side, given the remarkable success of DL algorithms, such as the case of CNNs in remote sensing, there has been a surge of interest in vision transformers over the past few years. This heightened interest is attributed to their utilization of attention mechanisms, which excel in extracting global contextual information compared to CNNs. The enhanced ability of vision transformers to capture a broader context makes them an appealing choice for various applications. One promising application is in the field of AVs which helps vehicles to better ‘understand’ their environment and make the right decisions especially in complex scenarios and in critical weather situations.

4. Deep transformer Q-Networks (DTQN)

The goal of this work is to enhance the performance of the DQN model by integrating both CNNs and transformer techniques. This improvement is crucial for AVs as it directly impacts their ability to comprehend their surroundings and make informed decisions. In this section, we will delve into the architecture of our innovative approach called DTQN, which is visually depicted in Fig. 1. At time t , the agent begins by exploring its environment, taking a random action a_t , receiving the state s_t , and corresponding reward r_t . It then transitions to the next state s_{t+1} , storing this interaction in a buffer as a tuple (s_t, a_t, s_{t+1}, r_t) . The model samples a random batch of states s_t from this buffer, which are passed through CNNs to extract essential features. These features are subsequently fed into a Transformer module. The model predicts nine potential actions and selects the one with the highest Q-value. This selected action is executed by sending control commands to the AV within its environment.

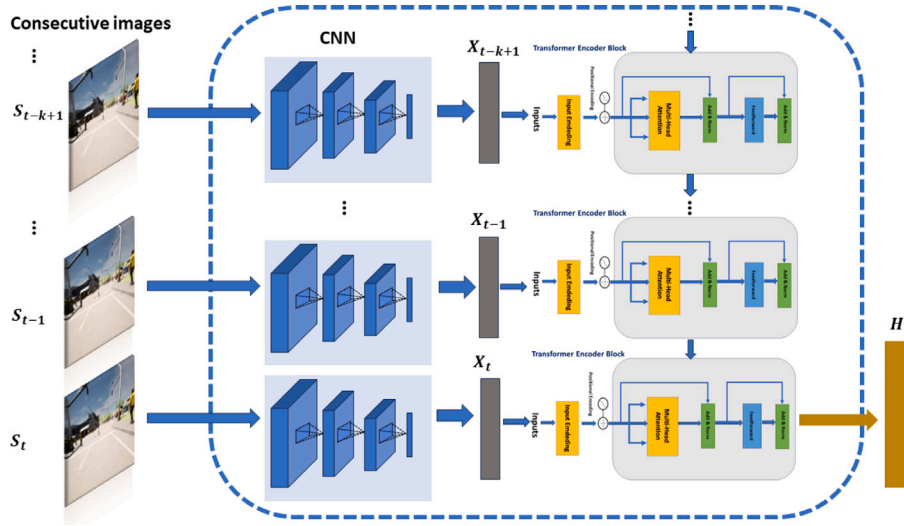


Fig. 3. The state network's architecture comprises two key components: CNNs for spatial feature extraction and a Transformer for capturing temporal features from the state data.

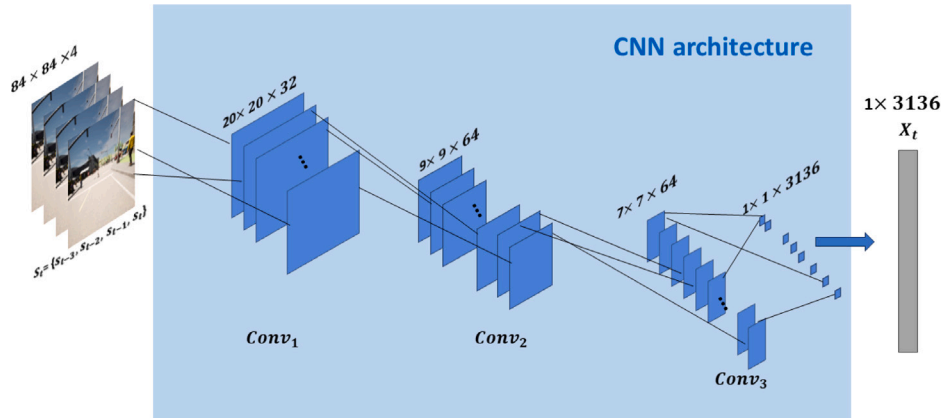


Fig. 4. The CNNs architecture comprises three convolutional layers that extract features from the input data, followed by a fully connected layer that prepares the data for input into a transformer model.

- **State space:** In the real world, just like human drivers use more than just their eyes to understand what is happening around them, AVs also rely on a sequence of images to perceive the movement of objects in their surroundings like human drivers. Because CNNs stand out at capturing spatial dependencies and Transformers excel at capturing temporal dependencies, we employ both a CNNs and a Transformer in designing our state encoder network to extract environmental states. The state network's design is illustrated in Fig. 3. At each time step t , we input the current image into a specialized CNNs to capture spatial characteristics, denoted as X_t . The latter will be used as an input to the transformer layer as depicted in Fig. 5. These spatial features from the current frame are then forwarded to a Transformer model.

The Transformer's role is to extract temporal relationships by analyzing how spatial features in the current frame are interconnected with those in the previous frames, thus capturing temporal dependencies. The Transformer generates the encoded state, denoted as H_t at each time step t , and this state is subsequently input into the policy network, which is the DQN used in our approach. We have designed a CNNs comprising three convolutional (Conv) layers. This CNNs is specifically tailored to extract essential spatial features from images captured by the front camera of the AV. The main architecture of our designed CNNs is represented in Fig. 4.

In this particular scenario, our model processes a sequence of four consecutive RGB images. Initially, these images have dimensions

of $800 \times 600 \times 3 \times 4$ pixels. However, they are resized to $84 \times 84 \times 3 \times 4$ pixels and then converted to grayscale. This transformation results in a new state, which is represented as $s_t \in \mathbb{R}^{84 \times 84 \times 4}$ before it is input into the neural network. The three convolutional (Conv) layers have kernel sizes of 8×8 , 4×4 , and 3×3 , and each layer uses 32, 32, and 64 kernels, respectively. Instead of using pooling, we apply strided convolutions with strides of 4, 2, and 1 in the three layers. This choice is made because strided convolutions are fully differentiable and enable the network to autonomously learn spatial downsampling.

In Fig. 4, there are three Conv layers with feature map dimensions of $20 \times 20 \times 32$, $9 \times 9 \times 64$, and $7 \times 7 \times 64$, respectively. The output is a vector, which leads to a final feature map size of 1×3136 , represented as $X_t \in \mathbb{R}^{1 \times 3136}$. To describe this, we can use the function F_{CNNs} , depicted as follows:

$$F_{CNNs}(s_t) = X_t \quad (2)$$

Transformers exhibit remarkable capabilities in enhancing feature extraction and capturing long-range dependencies within sequences. Unlike LSTM, which struggles with long-term dependencies due to the vanishing gradient problem, Transformers employ self-attention mechanisms. These mechanisms allow transformers to consider any position in the input sequence when making predictions. Consequently, we opt to utilize transformers as sub-networks in our approach (See Fig. 5). To this

end, we feed the resulting sequence of k vectors, denoted as $\{X_{t-k+1}, \dots, X_{t-1}, X_t\}$, into the transformer block. Each vector in this sequence is computed using Eq. (2) from the k successive states, which are $\{s_{t-k+1}, \dots, s_{t-1}, s_t\}$. In our specific case, we have chosen $k = 4$ to effectively learn the temporal dependence of four consecutive frames. The output of the Transformer is a vector that shares the same shape as H_t , which is represented as $X_t \in \mathbb{R}^{1 \times 3 \times 136}$ as shown in Fig. 3.

• **Action space:**

In the CARLA simulator, the AV's control system interacts with its environment by receiving three specific commands: acceleration, steering, and braking. These commands are expressed as numerical values: acceleration values fall within the range of $[0, 1]$, steering values within the range of $[-1, 1]$, and braking values within the range of $[0, 1]$.

Because the DQN is a discrete DRL algorithm, the agent must make discrete action choices as per Table 1. Therefore, during each time step (represented as t), the agent must select a specific action, which is essentially a combination of acceleration, steering, and brake commands. It is crucial to ensure that each of these commands adheres to its respective allowed range.

- **State Transition :** The transition probability from state s_t to the next state s_{t+1} is defined as:

$$s_{t+1} = F_{CNNs+Trans}(s_t, a_t) \quad (3)$$

- **Reward function :** Inspired by the book (Palanisamy, 2018), we have tailored our reward system to suit our specific situation, we create a reward system that considers various scenarios within city traffic. The primary objective is to ensure that the AV navigates without any accidents involving fellow road users like cars, bicycles, motorcycles, or pedestrians while successfully arriving at its intended destination. To achieve this, our reward system comprises four key components: an incentive for maintaining an appropriate speed, a penalty for collisions, any instances of the AV straying off the road or entering another lane, and the distance remaining to the destination.

In order to simplify, we represent the current distance to the destination as D_{cu} and the previous distance as D_{pre} . The AV's forward speed is denoted as V_{speed} , and its deviation from the correct lane is captured by two variables, $M_{offroad}$ and $M_{otherlane}$. In cases of collisions involving pedestrians, other vehicles, or any road obstacle, a penalty represented by $C_{collision}$ is applied, which adjusts the reward as R_{t1} . Additionally, we calculate a reward based on the change in distance to the goal: if the AV gets closer to the goal ($D_{pre} > D_{cu}$), it receives an increased reward, expressed as R_{t2} . This encourages the agent to progress towards the destination. The AV's current speed also contributes to its reward, with a maximum value of 3 (derived from 30 divided by 10), denoted as R_{t3} . This incentivizes the agent to maintain speed. Ensuring the AV stays within its designated lane is crucial, and we enforce this by imposing minor penalties when the agent deviates from its lane or enters another lane. These penalties are reflected in rewards R_{t4} and R_{t5} . Lastly, upon successfully reaching its intended destination, the agent receives a reward of $R_{t6} = 100$. This summarizes our reward function approach.

$$reward = \begin{cases} R_{t1} = -C_{collision} \\ R_{t2} = D_{pre} - D_{cu} \\ R_{t3} = \frac{V_{speed}}{10} \\ R_{t4} = -0.05 \times M_{offroad} \\ R_{t5} = -0.05 \times M_{otherlane} \\ R_{t6} = 100 \\ R_t = R_{t1} + R_{t2} + R_{t4} + R_{t5} + R_{t6} \end{cases} \quad V_{speed} \in [0, V_{limit}] \quad (4)$$

- **Training stage:** In our DTQN architecture, we employ a two-network approach, consisting of the policy network and the target network. This setup forms the foundation for training our RL agent to make informed decisions in dynamic environments.

At the outset of the RL process, the agent begins with an initial state denoted as s_t . It then takes an action, which is denoted as a_t , and advances to the subsequent state, which we represent as s_{t+1} . Simultaneously, the agent receives a reward, quantified as r_t . This crucial information is encapsulated within a tuple: (s_t, a_t, r_t, s_{t+1}) . These tuples are systematically collected and stored in a data structure known as a replay buffer. This buffer serves as a memory of past experiences, which the agent can draw upon during training.

As the training process unfolds, we sample random batches of 64 tuples from the replay buffer. These mini-batches are instrumental in training our neural networks efficiently. The policy network is primarily responsible for predicting Q-values, denoted as $Q(s_t, a_t)$, for various actions within our action set, which are represented as $\{a_0, a_1, \dots, a_8\}$. Meanwhile, the target network is tasked with predicting the Q-value $Q(s_{t+1}, a_{t+1})$ for the subsequent state-action pair.

Now, to imbue our RL agent with the ability to make optimal decisions, we employ the Bellman equation, represented as follows:

$$Q_*(s_t, a_t) = E[r_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})] \quad (5)$$

Here, γ is the discount factor, which signifies the importance of future rewards. The objective is to compute the optimal Q-value, denoted as $Q^*(s_t, a_t)$. This optimal value represents the highest expected cumulative reward the agent can obtain by taking action a_t in state s_t . The crux of our training process lies in minimizing the discrepancy between predicted Q-values and these optimal values.

To accomplish this, we employ the mean square error (MSE) loss function. It quantifies the disparity between our network's Q-value predictions and the optimal Q-values. Mathematically, this loss can be expressed as:

$$Loss = \frac{1}{N} \sum_{i=1}^N (Q(s_i, a_i) - Q^*(s_i, a_i))^2, \quad (6)$$

whereby N represents the batch size.

Lastly, the Adam optimizer comes into play to fine-tune the weights of our neural networks. It does so through the process of backpropagation, systematically adjusting the network parameters to minimize the MSE loss. This iterative optimization process is pivotal in enhancing the model's performance over time. The remaining training parameters are presented in the Table 2.

5. Simulation setup

We conducted AD experiments in a challenging intersection scenario using the CARLA simulator (version 0.9.10) and PyTorch. This specific scenario posed a significant test for AD systems due to its complex traffic dynamics.

In this scenario, vehicles have to navigate through multiple lanes while dealing with various challenges, such as handling oncoming traffic, merging with the flow of vehicles, and responding to pedestrians at designated crosswalks. The complexity arises from the multitude of real-time decisions and actions needed for safe and efficient navigation.

To make our simulation realistic and comprehensive, we included diverse traffic participants such as pedestrians, cars, bicycles, and motorcycles. This diverse set of road users allowed us to thoroughly evaluate how well our AD algorithm performed in a wide range of traffic situations and challenges.

To create demanding traffic conditions, we adjusted parameters like vehicle speeds, distances between vehicles, and the frequency of traffic

Transformer Encoder Block

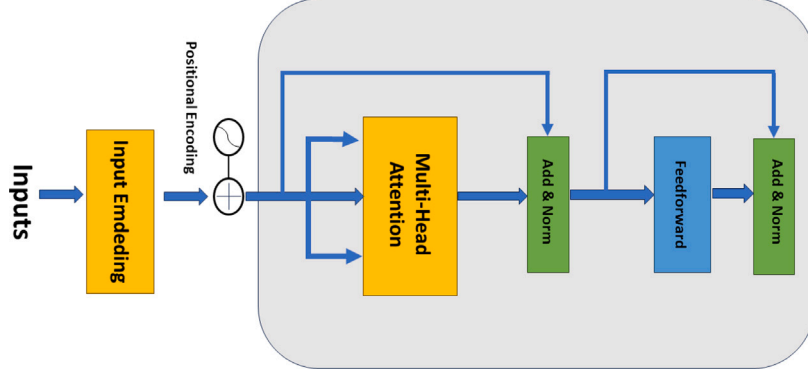


Fig. 5. The architecture of Transformer used in our DTQN model.

Table 1

Descriptions of actions and their corresponding values.

Action	Description	Control commands		
		Steering	Throttle	Brake
a_0	Cost	0.0	0.0	0.0
a_1	Turn left & Accelerate	-0.5	0.5	0.0
a_2	Turn right & Accelerate	0.5	0.5	0.0
a_3	Full forward	0.0	1.0	0.0
a_4	Brake	0.0	0.0	0.5
a_5	Slight turn left & Accelerate	0.0	0.5	-0.05
a_6	Slight turn right & Accelerate	0.0	0.5	0.05
a_7	Maintain forward	0.0	0.5	0.0
a_8	Accelerate moderately	0.0	0.6	0.0

Table 2

Simulation parameters.

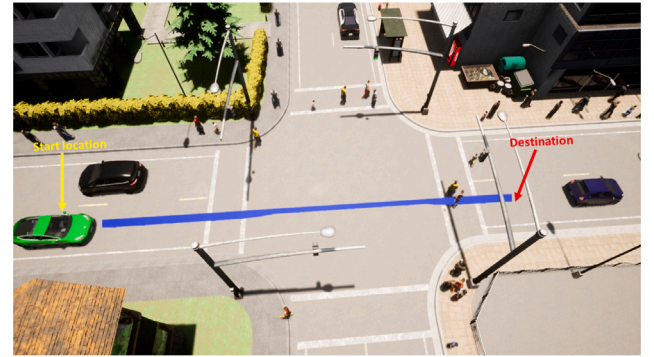
Parameter	Value
Learning rate	0.001
Number of episodes	3000
Batch size	64
Discount factor (γ)	0.99
ϵ_{start}	1
ϵ_{end}	0.01
Replay memory size	10000
Embedding dimension	3136
Number of head	4
Hidden dimension	512
$C_{\text{collision}}$	-100
V_{limit}	30 m/s

participants entering the simulation. This enabled us to subject our algorithms to conditions closely resembling real-world driving scenarios, which is crucial for assessing the effectiveness of our proposed solutions in complex driving environments. Additionally, we integrate CARLA with Gym, an open-source Python library developed by OpenAI (Brockman et al., 2016). Originally designed for the purpose of creating and assessing RL algorithms, Gym serves as a conduit for communication between the DRL algorithm and the environment.

In Fig. 6, the illustration of a densely populated traffic environment where an AV navigates safely through an intersection. The vehicle follows a predefined path, starting from an initial position and avoiding collisions with pedestrians, cyclists, motorcycles, and other vehicles. This scenario involves 200 vehicles moving randomly, including motorcycles, cyclists, and four-wheeled vehicles, all managed by the CARLA traffic manager. All vehicles operate in autopilot mode, maintaining a safe distance of 2.5 m between them and adhering to a speed limit of 30 m/s. Additionally, we randomly placed 100 pedestrians, with 80% of them using crosswalks to cross the road, adding to the complexity of the scenario.



(a)



(b)

Fig. 6. The proposed scenario: (a) The ego Vehicle deftly maneuvering through a congested intersection, and (b) The route an AV takes to smoothly reach its intended destination.

The simulator initially captures images at a size of 800×600 pixels. In order to reduce unnecessary data, we opt to capture 5 frames per second (fps) during both the training and testing phases. Training episodes conclude under the following conditions: (1) if there is a collision between the AV and other road users; (2) if the AV successfully reaches its destination; (3) if the episode surpasses the maximum number of training steps, which is set at 800.

6. Results

In this section, we demonstrate the efficacy of our proposed DTQN approach by showcasing the training process and testing results. We also provide a comparative analysis between DTQN and the DQN model to highlight the advantages of our approach.

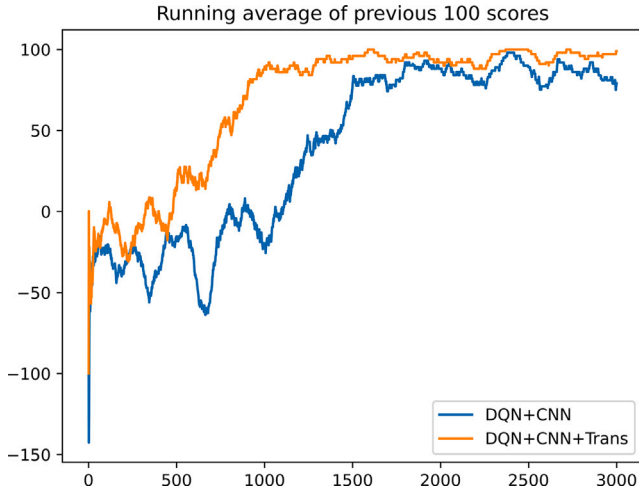


Fig. 7. The comparison between the average reward of *DQN + CNNs* and *DQN + CNNs + Trans* during episodes.

6.1. Training results

In this section, to assist the performances of DTQN method, we have implemented a basic DQN solution. We have performed a comparison between the two solutions to assist the performances of DTQN in the context of AV control. To assess the learning performance in RL, we often rely on rewards as a standard criterion. In Fig. 7, we present the average rewards attained during episodes.

The figure unequivocally demonstrates that the learning performance of the AV control approach employing DTQN surpasses that of the DQN method. This superiority is evident in the average reward trajectory. One key advantage of DTQN is its ability to learn and adapt more rapidly. This is due to the inherent strength of transformer models in capturing long-range dependencies within data. This capability empowers the AV to gain a deeper understanding of its environment, facilitating more accurate decision-making. Furthermore, it is noteworthy that the average reward of DTQN begins its ascent after just 500 episodes, while the DQN method requires twice as many episodes, specifically 1000, to exhibit similar progress. This early and swift improvement in reward is a clear indication of DTQN's efficacy.

Moreover, when we examine the convergence behavior, we find that the curve of the average reward for DTQN reaches a peak value around the 1500 episode. In contrast, DQN takes significantly longer, converging at around the 2500 episode. This discrepancy underscores that DTQN not only learns more efficiently but also achieves a higher level of performance in AV control.

The training results show that DTQN is a superior learning method compared to DQN for AVs control tasks, offering faster learning, better adaptation to the environment, and quicker convergence to optimal performance.

6.2. Testing results

After completing the training phase of our proposed approach, the next crucial step is to evaluate the performance of our model. To achieve this, we subject our model to testing across three distinct intersections as shown in Fig. 8. Notably, the first intersection mirrors the conditions encountered during the training phase, while the other two intersections differ from what the AV was exposed to during training. Furthermore, our evaluation strategy extends to assessing our model's performance under various traffic densities. These densities are determined by the number of pedestrians and vehicles present in the simulation scenario.

For the purposes of our evaluation, we deliberately selected five different density scenarios, each defined as follows: (1) $Ped = 50$, and $Veh = 50$; (2) $Ped = 100$, and $Veh = 100$; (3) $Ped = 140$, and $Veh = 140$; (4) $Ped = 200$, and $Veh = 200$; (5) $Ped = 220$, and $Veh = 220$.

Our evaluation procedure involves employing the trained policy network to control the AV as it navigates these three intersections. To gauge the effectiveness of our model, we employ two key metrics: travel delay and the number of collisions. These metrics provide valuable insights into the model's performance.

In each evaluation episode, we compute the travel delay and the number of collisions and subsequently determine the average values for both metrics across all episodes. These averages are calculated using the following equations:

$$Av_{travel} = \frac{1}{N} \sum_{i=1}^N Trav_i \quad (7)$$

$$Av_{collision} = \frac{1}{N} \sum_{i=1}^N Colis_i, \quad (8)$$

whereby Av_{travel} represents the average travel delay, and $Av_{collision}$ represents the average number of collisions. The variables $Trav_i$ and $Colis_i$ denote the travel delay and number of collisions in each specific episode, while N corresponds to the total number of episodes, which is consistent across all scenarios in our evaluation.

We conducted a comprehensive analysis in our study, repeatedly simulating each test scenario a total of 10 times. In doing so, we aimed to establish a robust understanding of the outcomes and determine confidence intervals for our results. These confidence intervals are visually represented in Fig. 9, which offers a clear overview of the travel delay and average collision number observed at each intersection.

To delve further into our findings, we present the specific results for the travel delay in each of the three test scenarios, illustrated in Figs. 9(a), 9(c), and 9(e), we delve deeper into the specifics of travel delay in each of the three test scenarios. Notably, in the first test, all travel delay values fall within the range of 5 to 7.5 s. As we progress through subsequent tests, we observe a gradual increase in travel delay. This can be attributed to the growing number of vehicles and pedestrians, causing congestion and delays at the intersections. A particularly noteworthy observation is that, across all tests conducted at each intersection, our DTQN model consistently outperforms the conventional DQN model. This observation underscores the remarkable efficiency and effectiveness of our DTQN model when compared to the standard DQN model.

In the provided figures, namely, Figs. 9(b), 9(d), and 9(f) we present the average collision data for three different intersections. Notably, our analysis reveals interesting insights.

Across all three tests, it is evident that both the DTQN and DQN models exhibit similar performance when it comes to average collision rates. However, a noteworthy divergence arises in the latter two tests, particularly when traffic volumes are significantly elevated.

In these high-traffic scenarios, the DTQN model significantly outperforms the DQN model in mitigating collisions with other road users. This performance disparity can be attributed to the transformer architecture employed in the DTQN. This architecture equips the model with the capability to leverage prior environmental information, enabling it to make more informed decisions and thus contributing to its superior collision reduction performance in high-traffic conditions.

This highlights the advantages introduced by the DTQN model. Unlike traditional DQN, which relies on a fixed-size experience replay buffer, DTQN employs a self-attention mechanism that allows it to effectively process and retain long-term dependencies. This enables the model to capture intricate traffic patterns, predict potential hazards more accurately, and adjust its actions accordingly. Moreover, the ability of DTQN to dynamically prioritize relevant past observations enhances its decision-making process, particularly in complex and rapidly changing environments. This is especially crucial in high-traffic

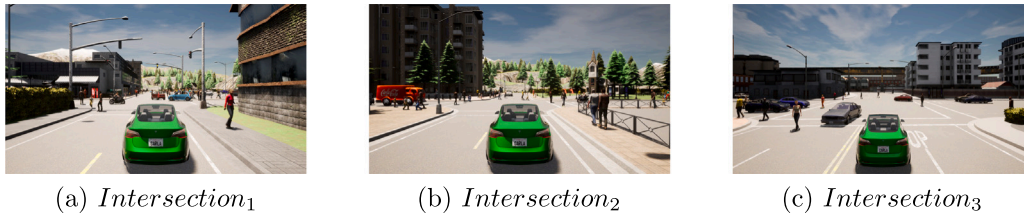


Fig. 8. The three snapshots of intersections where AV is tested: (a) The first snapshot captures an intersection located in Town 4, which is the same location used during the training phase for the AV; (b) In the second snapshot, another intersection in Town 4, distinct from the one mentioned in point (a); (c) The third snapshot shows an intersection that belongs to Town 3.

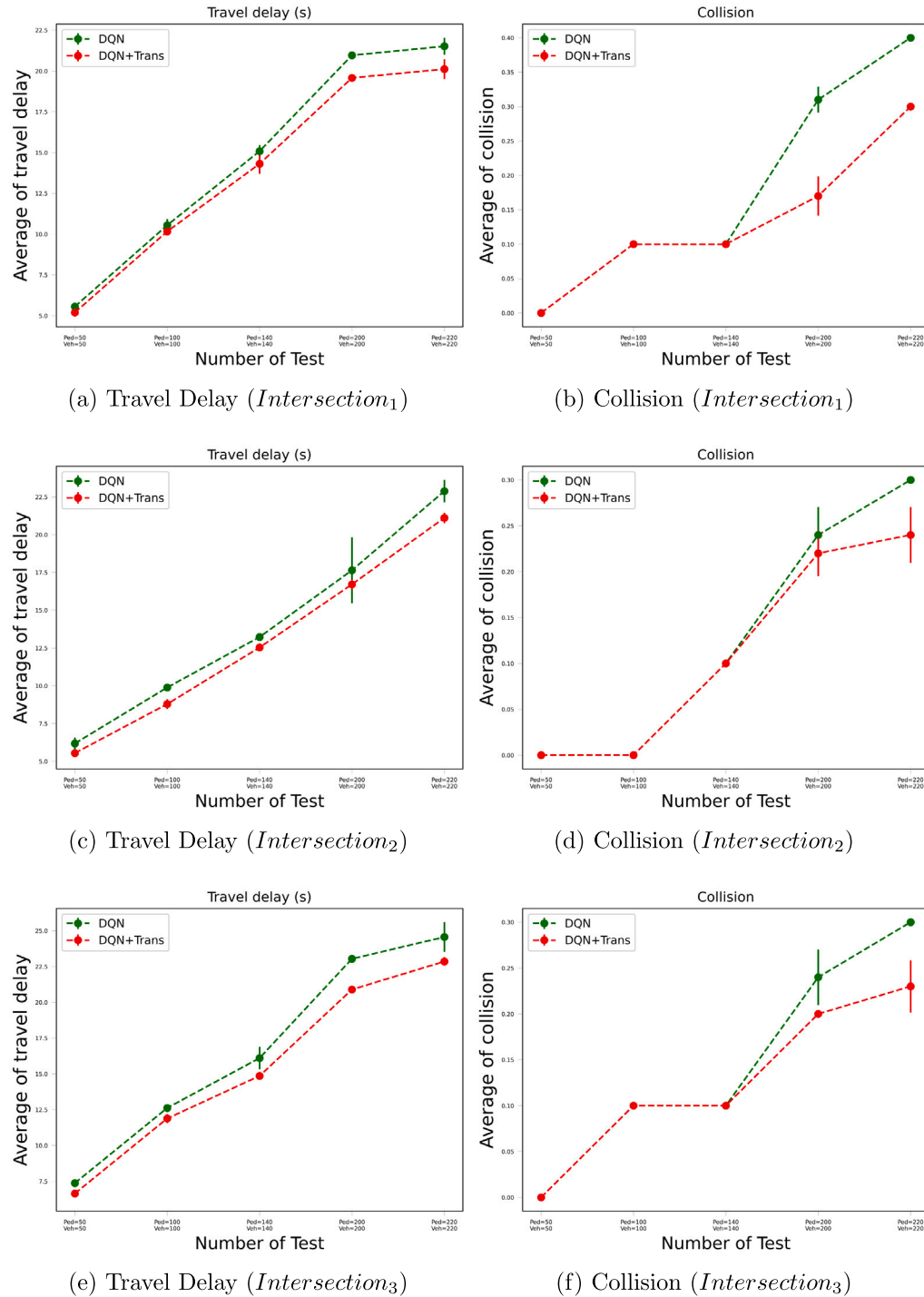


Fig. 9. Comparison of travel delay and number of collision in the first, second, and third intersections in each test.

scenarios where interactions between multiple agents become highly unpredictable. The improved performance of DTQN demonstrates the effectiveness of incorporating transformers into RL, paving the way for more robust and adaptive AD strategies.

7. Conclusion

This paper introduced a novel approach, the DTQN, aimed at enhancing the capabilities of CNNs by integrating them with transformer architectures. The DTQN model effectively extracted spatial features from images sourced from AVs while employing transformers to capture temporal dependencies among these features. This state vector was then fed into a policy network for predicting Q-values for various actions. Our model was specifically tailored for controlling AVs in complex intersection scenarios, replete with dense traffic and the presence of diverse road users such as pedestrians, cyclists, motorcycles, and other vehicles. To assess and validate our model, we leveraged the CARLA simulator, renowned for its highly realistic environments closely resembling the real world. The results of our experiments underscored the effectiveness of the DTQN model. It consistently outperformed traditional DQN that relied solely on CNNs, significantly reducing travel delays and the incidence of collisions.

Despite the great results achieved by our model, the DTQN model is considered a discrete algorithm because it relies on discrete actions. As a result, it is limited by the number of actions provided in Table 1. In our future work, we aim to explore continuous DRL algorithms to offer AVs greater flexibility in navigating their environment. Additionally, we plan to integrate explainable DRL to ensure that the decision-making process is interpretable, allowing us to identify and mitigate potential risks and avoid unsafe events. Although ride comfort and energy efficiency are critical considerations in the AV domain, they fall outside the scope of the present study; however, we plan to incorporate them in future research. Furthermore, we will address the challenge of navigating AVs through intersections with heavy traffic, including making left and right turns.

CRedit authorship contribution statement

Badr Ben Elallid: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Methodology, Investigation, Funding acquisition, Formal analysis. **Nabil Benamar:** Writing – review & editing, Writing – original draft, Supervision. **Miloud Bagaa:** Writing – review & editing, Validation, Supervision. **Nabil Mrani:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Ahmed, M., Lim, C.P., Nahavandi, S., 2021. A deep q-network reinforcement learning-based model for autonomous driving. In: 2021 IEEE International Conference on Systems, Man, and Cybernetics. SMC, IEEE, pp. 739–744.
- Bao, Z., Hossain, S., Lang, H., Lin, X., 2023. A review of high-definition map creation methods for autonomous driving. *Eng. Appl. Artif. Intell.* 122, 106125.
- Be, 2021. Road traffic injuries. <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W., 2016. Openai gym. *arXiv preprint arXiv:1606.01540*.
- Chen, L., Hu, X., Tang, B., Cheng, Y., 2020. Conditional DQN-based motion planning with fuzzy logic for autonomous driving. *IEEE Trans. Intell. Transp. Syst.* 23 (4), 2966–2977.
- Chen, S., Sun, Y., Li, D., Wang, Q., Hao, Q., Sifakis, J., 2022. Runtime safety assurance for learning-enabled control of autonomous driving vehicles. In: 2022 International Conference on Robotics and Automation. ICRA, IEEE, pp. 8978–8984.
- Chen, L., Wu, P., Chitta, K., Jaeger, B., Geiger, A., Li, H., 2024. End-to-end autonomous driving: Challenges and frontiers. *IEEE Trans. Pattern Anal. Mach. Intell.*
- Constantinides, P., Monteiro, E., Mathiasen, L., 2024. Human-AI joint task performance: Learning from uncertainty in autonomous driving systems. *Inf. Organ.* 34 (2), 100502.
- Du, Y., Chen, J., Zhao, C., Liu, C., Liao, F., Chan, C.-Y., 2022. Comfortable and energy-efficient speed control of autonomous vehicles on rough pavements using deep reinforcement learning. *Transp. Res. Part C: Emerg. Technol.* 134, 103489.
- El Hamdani, S., Benamar, N., Younis, M., 2020a. Pedestrian support in intelligent transportation systems: challenges, solutions and open issues. *Transp. Res. Part C: Emerg. Technol.* 121, 102856.
- El Hamdani, S., Benamar, N., Younis, M., 2020b. A protocol for pedestrian crossing and increased vehicular flow in smart cities. *J. Intell. Transp. Syst.* 24 (5), 514–533.
- Elallid, B.B., Abouamar, A., Benamar, N., Kobbane, A., 2023. Vehicles control: Collision avoidance using federated deep reinforcement learning. In: GLOBECOM 2023-2023 IEEE Global Communications Conference. IEEE, pp. 4369–4374.
- Elallid, B.B., Bagaa, M., Benamar, N., Mrani, N., 2023b. A reinforcement learning based approach for controlling autonomous vehicles in complex scenarios. In: 2023 International Wireless Communications and Mobile Computing. IWCMC, IEEE, pp. 1358–1364.
- Elallid, B.B., Benamar, N., Hafid, A.S., Rachidi, T., Mrani, N., 2022a. A comprehensive survey on the application of deep and reinforcement learning approaches in autonomous driving. *J. King Saud Univ.- Comput. Inf. Sci.*
- Elallid, B.B., Benamar, N., Mrani, N., Rachidi, T., 2022b. DQN-based reinforcement learning for vehicle control of autonomous vehicles interacting with pedestrians. In: 2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies. 3ICT, IEEE, pp. 489–493.
- Elallid, B.B., Hamdani, S.E., Benamar, N., Mrani, N., 2022c. Deep learning-based modeling of pedestrian perception and decision-making in refuge island for autonomous driving. In: Computational Intelligence in Recent Communication Networks. Springer, pp. 135–146.
- Esslinger, K., Platt, R., Amato, C., 2022. Deep transformer q-networks for partially observable reinforcement learning. *arXiv preprint arXiv:2206.01078*.
- Guo, K., Jing, W., Chen, J., Pan, J., 2023. CCIL: Context-conditioned imitation learning for urban driving. *arXiv preprint arXiv:2305.02649*.
- Hu, J., Kong, H., Zhang, Q., Liu, R., 2022. Enhancing scene understanding based on deep learning for end-to-end autonomous driving. *Eng. Appl. Artif. Intell.* 116, 105474.
- Huang, Z., Liu, H., Wu, J., Lv, C., 2023. Conditional predictive behavior planning with inverse reinforcement learning for human-like autonomous driving. *IEEE Trans. Intell. Transp. Syst.*
- Lamssaggad, A., Benamar, N., Hafid, A.S., Msahli, M., 2021. A survey on the current security landscape of intelligent transportation systems. *IEEE Access* 9, 9180–9208.
- Li, G., Lin, S., Li, S., Qu, X., 2022. Learning automated driving in complex intersection scenarios based on camera sensors: A deep reinforcement learning approach. *IEEE Sens. J.* 22 (5), 4687–4696.
- Li, D., Liu, B., Huang, Z., Hao, Q., Zhao, D., Tian, B., 2023. Safe motion planning for autonomous vehicles by quantifying uncertainties of deep learning-enabled environment perception. *IEEE Trans. Intell. Veh.* 9 (1), 2318–2332.
- Nugroho, S., Setiadi, D.R.I.M., Islam, H.M.M., 2024. Exploring DQN-based reinforcement learning in autonomous highway navigation performance under high-traffic conditions. *J. Comput. Theor. Appl.* 1 (3), 274–286.
- Palanisamy, P., 2018. Hands-on Intelligent Agents with OpenAI Gym: Your Guide to Developing AI Agents using Deep Reinforcement Learning. Packt Publishing.
- Pérez-Gil, O., Barea, R., López-Guillén, E., Bergasa, L.M., Revenga, P.A., Gutiérrez, R., Díaz, A., 2021. Dqn-based deep reinforcement learning for autonomous driving. In: Advances in Physical Agents II: Proceedings of the 21st International Workshop of Physical Agents (WAF 2020), November 19–20, 2020, Alcalá de Henares, Madrid, Spain. Springer, pp. 60–76.
- Pravallika, A., Hashmi, M.F., Gupta, A., 2024. Deep learning frontiers in 3D object detection: A comprehensive review for autonomous driving. *IEEE Access*.
- Selvaraj, D.C., Hegde, S., Amati, N., Deflorio, F., Chiasserini, C.F., 2023. A deep reinforcement learning approach for efficient, safe and comfortable driving. *Appl. Sci.* 13 (9), 5272.
- Shin, J., Badgwell, T.A., Liu, K.-H., Lee, J.H., 2019. Reinforcement learning—overview of recent progress and implications for process control. *Comput. Chem. Eng.* 127, 282–294.
- Sivayazi, K., Mannayee, G., 2024. Modeling and simulation of a double DQN algorithm for dynamic obstacle avoidance in autonomous vehicle navigation. *E-Prime-Adv. Electr. Eng. Electron. Energy* 8, 100581.

- Stigall, W.A., 2024. Transforming game play: A comparative study of DCQN and dtqn architectures in reinforcement learning. *arXiv preprint arXiv:2410.10660*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30.
- Wang, X., Althoff, M., 2023. Safe reinforcement learning for automated vehicles via online reachability analysis. *IEEE Trans. Intell. Veh.*.
- Wang, J., Hu, C., Zhao, J., Zhang, L., Han, Y., 2024. Deep Q-network-enabled platoon merging approach for autonomous vehicles. *Transp. Res. Rec.* 2678 (7), 17–31.
- Wu, J., Huang, Z., Hu, Z., Lv, C., 2023a. Toward human-in-the-loop AI: Enhancing deep reinforcement learning via real-time human guidance for autonomous driving. *Engineering* 21, 75–91.
- Wu, J., Song, Z., Lv, C., 2023b. Deep reinforcement learning-based energy-efficient decision-making for autonomous electric vehicle in dynamic traffic environments. *IEEE Trans. Transp. Electr.* 10 (1), 875–887.
- Zhang, D., Tao, Z.-Y., Zhou, K., Zhou, F.-R., Peng, Q.-Y., Tang, Y.-Y., 2024. Improving the energy efficiency and riding comfort of high-speed trains across slopes by the optimized suspension control. *Energy* 307, 132660.